

Lab 13

Lab Manual: Implementation of the Concepts of Exception Handling in Java

Lab Objective:

To enable students to:

1. Understand the concept of exceptions and types in Java
2. Learn the complete structure of exception handling
3. Differentiate between built-in and user-defined exceptions
4. Practice writing robust Java programs using real-life examples
5. Gain confidence in writing reusable code

Theoretical Overview

1. What is an Exception?

An **exception** is an abnormal event that occurs during the execution of a program. It disrupts the normal flow of the program and can cause it to terminate if not handled properly

2. Types of Exceptions

Type	Description	Examples
Checked Exceptions	Caught during compile time	IOException, SQLException, FileNotFoundException
Unchecked Exceptions	Caught during runtime	ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException
Errors	Irrecoverable problems (not exceptions)	OutOfMemoryError, StackOverflowError

3. Why Use Exception Handling?

- 1) Prevents abrupt program termination
- 2) Gives user-friendly error messages
- 3) Helps in separating error-handling code from regular code
- 4) Useful in debugging and maintenance

4. Java Exception Handling Keywords

Keyword	Description
try	Wraps code that might throw an exception
catch	Catches and handles the exception

finally	Always runs
throw	Used to explicitly throw an exception
throws	Used to declare exceptions a method may throw

Question 1: ATM Withdrawal System

Create a program that simulates an ATM withdrawal. The program should throw an error if the user tries to withdraw more money than is available.

Code:

```
public class ATMWithdrawal {
    public static void main(String[] args) {
        int balance = 5000;           // Current balance in ATM account
        int withdrawAmount = 7000;    // Amount user wants to withdraw

        try {
            // Check if withdrawAmount is greater than balance
            if (withdrawAmount > balance) {
                // Manually throw an ArithmeticException with a custom message
                throw new ArithmeticException("Insufficient Balance!");
            }
            // Deduct the withdrawAmount from balance if funds are sufficient
            balance -= withdrawAmount;
            // Inform user about successful withdrawal and updated balance
            System.out.println("Withdrawal successful. Remaining balance: " + balance);
        } catch (ArithmeticException e) {
            // Catch block handles the exception thrown above
            // Prints the error message to the user
            System.out.println("Transaction Failed: " + e.getMessage());
        }
        // This statement executes regardless of exception
        System.out.println("Thank you for using our ATM.");
    }
}
```

Output:

Transaction Failed: Insufficient Balance!

Thank you for using our ATM.

Question 2: File Handling with Checked Exception

Read a file and display its content. If the file doesn't exist, show a custom message.

Code:

```
import java.io.*; // Import all classes for input/output

public class FileReading {
    public static void main(String[] args) {
        try {
            // Create BufferedReader object to read the file named "example.txt"
            BufferedReader reader = new BufferedReader(new FileReader("example.txt"));
            String line; // Variable to hold each line read from file
            // Read line by line until no more lines (null) are found
            while ((line = reader.readLine()) != null) {
                System.out.println(line); // Print each line on the console
            }
            reader.close(); // Close the file to free resources
        } catch (FileNotFoundException e) {
            // Handles case where file is not found or does not exist
            System.out.println("Error: File not found.");
        } catch (IOException e) {
            // Handles other IO exceptions such as read errors
            System.out.println("Error reading the file.");
        } finally {
            // This block always executes whether exception occurs or not
            System.out.println("File reading process completed.");
        }
    }
}
```

Output:

Error: File not found.

File reading process completed.

Question 3: Custom Exception, Age Check for Voting

If the user's age is below 18, throw a custom exception stating that they are not eligible to vote.

Code:

```
// Define a custom exception class by extending Exception
class AgeNotValidException extends Exception {
    // Constructor to initialize exception message
    public AgeNotValidException(String message) {
        super(message); // Pass message to Exception superclass
    }
}

public class VotingSystem {
    public static void main(String[] args) {
        int age = 15; // Example age of user trying to vote
        try {
            // Check if age is less than 18
            if (age < 18) {
                // Throw custom exception if user is not eligible
                throw new AgeNotValidException("You are not eligible to vote.");
            }
            // If age >= 18, print eligibility message
            System.out.println("You are eligible to vote!");
        } catch (AgeNotValidException e) {
            // Catch the custom exception and print the error message
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Real-World Scenario 4: Divide Two Numbers with Exception Handling

Problem:

Accept two numbers and perform division. Handle division by zero and invalid inputs using proper exception handling.

Code:

```
import java.util.Scanner;           // Import Scanner class to take input
import java.util.InputMismatchException; // Import exception class for invalid input

public class DivisionExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // Create Scanner object to read input
        try {
            System.out.print("Enter numerator: ");
            int a = sc.nextInt();           // Read numerator from user
            System.out.print("Enter denominator: ");
            int b = sc.nextInt();           // Read denominator from user
            int result = a / b;              // Perform division operation
            System.out.println("Result: " + result); // Print division result
        } catch (ArithmeticException e) {
            // Handles division by zero exception
            System.out.println("Cannot divide by zero.");
        } catch (InputMismatchException e) {
            // Handles invalid input type exception (e.g., entering a string)
            System.out.println("Please enter valid integers only.");
        } finally {
            sc.close();                     // Close Scanner to release resource
            System.out.println("Operation complete."); // Always executes
        }
    }
}
```